Agentic AI

Cahier des charges

Agentic analyst

Adnane Qassiri

2026

Table des matières

1 AI Analytics Agent – Cahier des charges **2**
- 1.1 Contexte et objectif . . . 2
- 1.2 Contexte métier . . . 2
- 1.3 Architecture fonctionnelle . . . 2
- 1.4 Agentic AI . . . 3
  - 1.4.1 Tool Use . . . 3
  - 1.4.2 Reflection . . . 3
- 1.5 RAG et compréhension de la base . . . 3
- 1.6 Génération et exécution contrôlée du SQL . . . 3
- 1.7 Analyse et visualisation . . . 4
- 1.8 Backend . . . 4
- 1.9 Déploiement et CI/CD . . . 4
- 1.10 Évaluation . . . 4
- 1.11 Stack technique . . . 5

1 AI Analytics Agent – Cahier des charges

1.1 Contexte et objectif

L'objectif du projet est de développer un **agent analytique intelligent** capable d'interagir avec une base de données PostgreSQL à partir de questions formulées en langage naturel.

Le système doit permettre à un utilisateur non technique d'obtenir automatiquement des analyses métier, des indicateurs et des visualisations sans avoir à écrire directement des requêtes SQL.

L'agent doit notamment être capable de :

— comprendre l'intention de la question utilisateur ;

— récupérer les informations pertinentes concernant le schéma et la documentation métier ;

— générer automatiquement des requêtes SQL ;

— valider et exécuter ces requêtes de manière contrôlée sur PostgreSQL ;

— analyser les résultats obtenus ;

— améliorer automatiquement les requêtes ou l'analyse lorsque les résultats sont insuffisants ;

— générer des visualisations adaptées ;

— produire une réponse finale synthétique et contextualisée.

1.2 Contexte métier

Le système sera développé autour d'une base de données relationnelle de type **e-commerce / retail** contenant plusieurs tables, par exemple :

— `customers` ;

— `orders` ;

— `order_items` ;

— `products` ;

— `payments`.

La base permettra de répondre à des questions analytiques telles que :

— Quels produits génèrent le plus de chiffre d'affaires ?

— Quelle est l'évolution mensuelle des ventes ?

— Quelles catégories expliquent une baisse du chiffre d'affaires ?

— Quelles régions présentent les meilleures performances ?

— Quel est le panier moyen par segment client ?

1.3 Architecture fonctionnelle

Le workflow principal du système sera le suivant :

1. Réception de la question utilisateur ;
2. Analyse de l'intention ;
3. Récupération du contexte pertinent sur la base de données ;
4. Génération de la requête SQL ;
5. Validation et contrôle de sécurité ;
6. Exécution de la requête sur PostgreSQL ;
7. Analyse des données retournées ;
8. Étape de réflexion afin d'évaluer la qualité du résultat ;
9. Amélioration éventuelle de la requête ou de l'analyse ;
10. Génération éventuelle d'une visualisation ;
11. Production de la réponse finale.

1.4 Agentic AI

L'orchestration sera réalisée avec **LangGraph** en utilisant principalement les patterns agentiques suivants :

1.4.1 Tool Use

L'agent disposera de plusieurs outils spécialisés, notamment :

— récupération du schéma PostgreSQL ;

— recherche dans la documentation métier ;

— exécution de requêtes SQL ;

— analyse des résultats avec Python ;

— génération de visualisations.

L'agent pourra sélectionner dynamiquement les outils nécessaires selon la question de l'utilisateur.

1.4.2 Reflection

Une étape de **Reflection** permettra à l'agent d'évaluer les résultats obtenus avant de produire la réponse finale.

Cette étape permettra notamment de détecter :

— une requête SQL incorrecte ou incomplète ;

— un résultat vide ou incohérent ;

— une analyse ne répondant pas complètement à la question utilisateur ;

— la nécessité d'effectuer une analyse supplémentaire ;

— la nécessité de modifier une requête ou une visualisation.

Lorsque cela est nécessaire, l'agent pourra corriger ou reformuler la requête SQL puis l'exécuter à nouveau.

1.5 RAG et compréhension de la base

Un système **RAG** sera utilisé afin de fournir à l'agent les informations nécessaires sur la base de données.

Les documents indexés pourront notamment contenir :

— description des tables ;

— description des colonnes ;

— relations entre les différentes tables ;

— définition des indicateurs métier ;

— règles de calcul des KPIs ;

— exemples de requêtes SQL.

Les embeddings et les documents seront stockés dans **PostgreSQL avec pgvector**.

1.6 Génération et exécution contrôlée du SQL

L'agent devra générer automatiquement les requêtes SQL nécessaires pour répondre aux questions des utilisateurs.

Avant leur exécution, les requêtes devront passer par une étape de validation.

Les principales règles de sécurité seront :

— connexion PostgreSQL en mode **read-only** ;

— autorisation uniquement des requêtes de lecture ;

— blocage des opérations `DELETE`, `UPDATE`, `INSERT`, `DROP` et `ALTER` ;

— limitation du volume de données retourné ;

— définition d'un timeout d'exécution ;

— validation de la requête avant exécution.

1.7 Analyse et visualisation

Après l'exécution du SQL, l'agent devra pouvoir analyser les données retournées afin d'identifier :

— tendances ;

— variations ;

— comparaisons ;

— contributions à une hausse ou une baisse ;

— KPIs principaux.

L'agent pourra également générer automatiquement une visualisation adaptée au résultat, par exemple :

— graphique en barres ;

— courbe temporelle ;

— graphique de comparaison ;

— nuage de points.

1.8 Backend

Le système sera exposé à travers une API développée avec **FastAPI**.
Les endpoints principaux pourront inclure :

— `POST /query` : envoi d'une question à l'agent ;

— `GET /health` : vérification de l'état du service ;

— `GET /sessions/{id}` : récupération d'une conversation ou analyse précédente.

1.9 Déploiement et CI/CD

L'application devra être conçue comme un système **end-to-end orienté production**.
Les différents composants seront conteneurisés avec **Docker**.
Le déploiement cible sera réalisé sur **AWS EC2**.
Un pipeline **CI/CD avec GitHub Actions** automatisera notamment :

1. l'exécution des tests ;
2. la construction de l'image Docker ;
3. le push de l'image vers un registre Docker ;
4. le déploiement de la nouvelle version sur EC2.

1.10 Évaluation

Une base de questions de référence sera créée afin d'évaluer le comportement de l'agent.
Chaque exemple pourra contenir :

— une question en langage naturel ;

— la requête SQL attendue ou le résultat attendu ;

— les informations nécessaires dans la réponse finale.

Les métriques pourront inclure :

— taux de requêtes SQL exécutées avec succès ;
— précision des requêtes SQL générées ;
— exactitude des réponses ;
— taux de blocage des requêtes non autorisées ;
— latence moyenne du système.

1.11 Stack technique

— **Agentic AI :** LangGraph, LLM, Tool Calling, Reflection ;
— **RAG :** Embeddings, PostgreSQL, pgvector ;
— **Data :** PostgreSQL, Pandas ;
— **Visualisation :** Matplotlib ou Plotly ;
— **Backend :** FastAPI ;
— **Infrastructure :** Docker, Docker Compose, AWS EC2 ;
— **CI/CD :** GitHub Actions.